Deliverable 1

Viktor M.L. Hedman og Adrian N. Vangen

Importent imports

In [51]:
```
import yaml #Imports the yaml library to writa yaml files
import pandas as pd #import pandas library for data analysis
import matplotlib.pyplot as plt #import matplotlib library for data visualizatio
import requests #import requests library for making HTTP requests
import math #Imports the math library to do som simple calculations an checks
```

Collecting the meteorological data for 2025

In [52]:
```
#Open the file containing the Frost Met ID and read the ID from the file
with open('Frost_Met_ID.txt', 'r') as f:
    Frost_Met_ID = f.readlines()[0].strip().split(':')[1]

#Set the endpoint and parameters for the API request
endpoint = 'https://frost.met.no/observations/v0.jsonld'
# Set the parameters for the API request
parameters = {
    'sources': 'SN17850',
    'elements': 'mean(air_temperature P1D)',
    'referencetime': '2025-01-01/2025-12-31',
}

def get_met_data(endpoint: str, parameters: dict, Frost_Met_ID: str) -> pd.DataF
    """This function retrives the average air temprature data
    from the frost.met.no API using the provided
    endpoint, parameters, and Frost Met ID.
    It makes an HTTP GET request to the API using the requests library.
    Stores the data in a json file and transforms it into
    a pandas DataFrame using the pandas library and a loop.
    It then returns a pandas DataFrame containing the retrieved data.
    """
    #Make the API request to retrieve temperature data.
    request = requests.get(endpoint, params=parameters, auth=(Frost_Met_ID, ''))

    #Check the status code of the response and handle errors if any.
    json=request.json()

    #Check if the request was successful and retrieve the data if it wa.
    if request.status_code == 200:
        data = json['data']
        print('Data retrieved from frost.met.no!')
    else:
        print('Error! Returned status code %s' % request.status_code)
        print('Message: %s' % json['error']['message'])
        print('Reason: %s' % json['error']['reason'])

    #Transform the data into a pandas DataFrame for analysis and visualization.
    df = pd.DataFrame()
    for i in range(len(data)):
        row = pd.DataFrame(data[i]['observations'])
        row['referenceTime'] = data[i]['referenceTime']
        row['sourceId'] = data[i]['sourceId']
```

```
        df = pd.concat([df, row])

    #Set the index of the DataFrame to the right index for easier analysis.
    df = df.reset_index()
    #Convert the 'referenceTime' column to a datetime object for easier analysis
    df['referenceTime'] = pd.to_datetime(df['referenceTime'])
    return df

#Use the get_met_data function to retrieve the temperature data and store it in
temp_df = get_met_data(endpoint, parameters, Frost_Met_ID)

#Show the first 5 rows and last 5 rows of the DataFrame to verify the data has b
temp_df.head(5)
```

```
Data retrieved from frost.met.no!
```

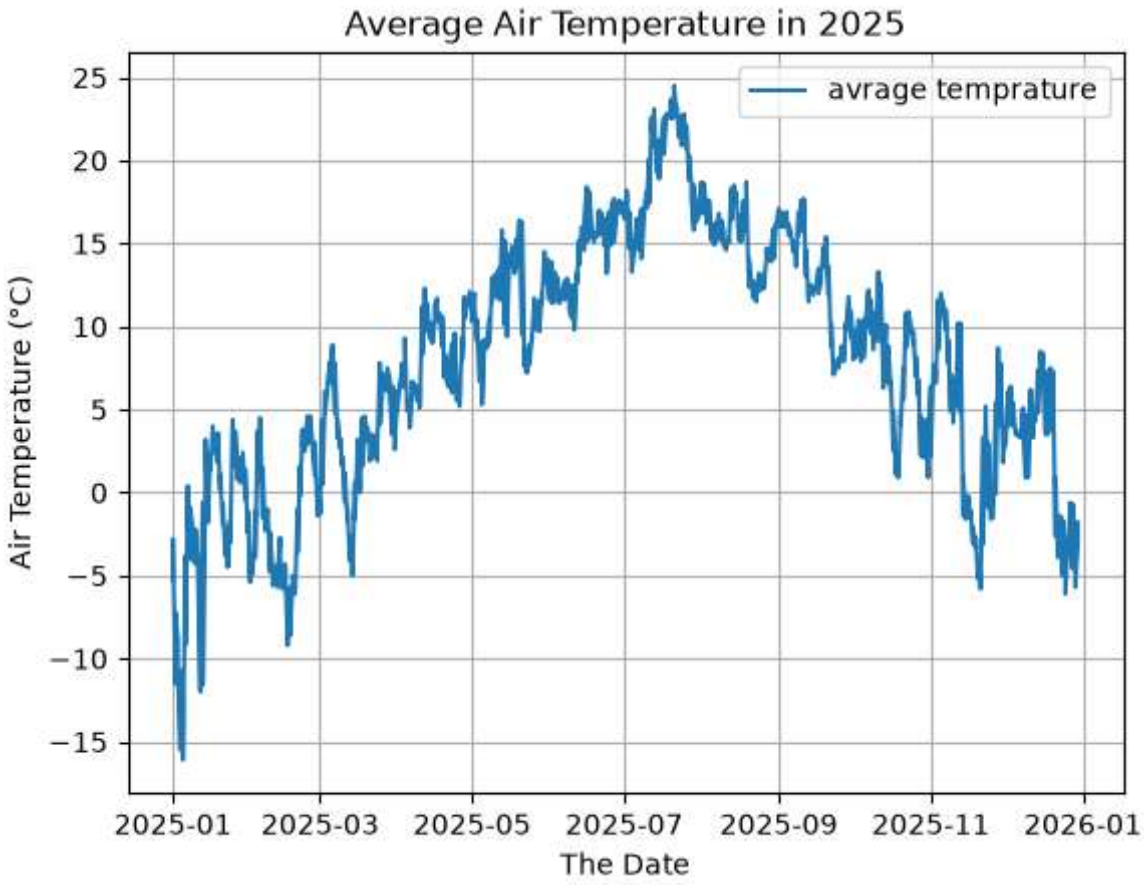
Out[52]:

	index	elementId	value	unit	level	timeOffset	timeRes
0	0	mean(air_temperature P1D)	-5.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	
1	1	mean(air_temperature P1D)	-2.8	degC	{'levelType': 'height_above_ground', 'unit': '...	PT6H	
2	0	mean(air_temperature P1D)	-11.5	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	
3	1	mean(air_temperature P1D)	-7.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT6H	
4	0	mean(air_temperature P1D)	-8.9	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	

Plots

In [53]:

```
#Plot the average air temperature data for 2025 using matplotlib library.
plt.plot(temp_df['referenceTime'], temp_df['value'], label='avrage temprature')
plt.title('Average Air Temperature in 2025')
plt.xlabel('The Date')
plt.ylabel('Air Temperature (°C)')
plt.legend()
plt.grid()
plt.show()
```



Summary table

In [54]:
```
#Calculate the mean, median, min, and max of the average air temperature data fo
Mean_air_temperature = temp_df[temp_df['elementId'] == 'mean(air_temperature P1D
Median_air_temperature = temp_df[temp_df['elementId'] == 'mean(air_temperature P
Min_air_temperature = temp_df[temp_df['elementId'] == 'mean(air_temperature P1D)
Max_air_temperature = temp_df[temp_df['elementId'] == 'mean(air_temperature P1D)

#Store the calculated values in a pandas DataFrame.
df_summery = pd.DataFrame({'Degrees': [Mean_air_temperature, Median_air_temperat
#print the DataFrame containing the calculated values.
df_summery['Degrees']=df_summery['Degrees'].map('{:.2f}'.format)
#We retrieve daily mean temperatures.
print(df_summery)
```

```
                        Degrees
Mean Air Temperature:      7.90
Median Air Temperature:    8.35
Min Air Temperature:     -16.10
Max Air Temperature:      24.50
```

The graf is ploted nicly with titels and labels. We see that the data is writen in a table and the values are resanebal compered to the graf.

Task 2 Requests and request precipitation data

In [55]:
```
#define the parameters for the API request to retrieve precipitation data.
parameters_p = {
    'sources': 'SN17850',
    'elements': 'sum(precipitation_amount P1D)',
    'referencetime': '2025-01-01/2025-12-31',
```

```
}

#Use the get_met_data function to retrieve the precipitation data and store it i
precipitation_df = get_met_data(endpoint, parameters_p, Frost_Met_ID)

#Show the first 5 rows and last 5 rows of the precipitation DataFrame to verify
precipitation_df.head(5)
```

```
Data retrieved from frost.met.no!
```

Out[55]:

	index	elementId	value	unit	timeOffset	timeResolution	timeSeriesId
0	0	sum(precipitation_amount P1D)	5.4	mm	PT18H	P1D	(
1	1	sum(precipitation_amount P1D)	6.6	mm	PT6H	P1D	(
2	0	sum(precipitation_amount P1D)	0.0	mm	PT18H	P1D	(
3	1	sum(precipitation_amount P1D)	1.9	mm	PT6H	P1D	(
4	0	sum(precipitation_amount P1D)	0.1	mm	PT18H	P1D	(

We se that data was retrived nicly.

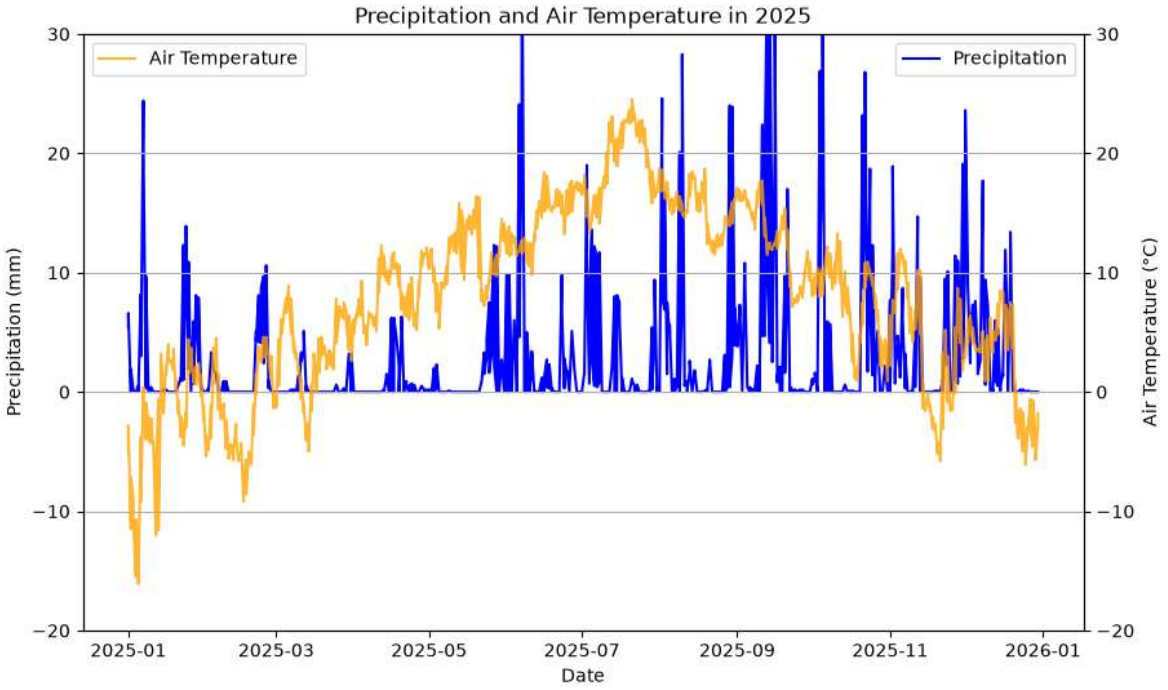
Plots percipotasjon and mean temprature

In [56]:

```
#Plot the precipitation and air temperature data for 2025 using matplotlib libra
fig, ax = plt.subplots(figsize=(10, 6))
#Plots the percipitation with labels.
ax.plot(precipitation_df['referenceTime'], precipitation_df['value'], label='Pre
ax.set_ylabel('Precipitation (mm)')
ax.set_xlabel('Date')
ax.legend(loc='upper right')
ax.set_ylim(-20,30)#Limits the y-axis

#Setts the x-axis to be the same as the persipitation.
ax=ax.twinx()
#Plots the temprature with labels.
ax.plot(temp_df['referenceTime'], temp_df['value'], label='Air Temperature', col
ax.set_ylabel('Air Temperature (°C)')
ax.set_title('Precipitation and Air Temperature in 2025')#Plots the title
ax.set_ylim(-20,30)#Limits the y-axis
ax.legend(loc='upper left')
plt.grid()

plt.show()
```



After som thoughts and testing we concluded that normal lineplots looked the best and gived the most usefull information. The y-axis must have the same starting point til give the most useful information. The y-lim is chosen by the max en minimum value of the air temprature.

Task 3

In [57]:
```
#define the parameters for the API request to retrieve daily mean temperature da
parameters_1925 = {
    'sources': 'SN17850',
    'elements':'mean(air_temperature P1D)',
    'referencetime': '1925-01-01/1925-12-31',
}

#Use the get_met_data function to retrieve the average temperature data from 192
df_1925 = get_met_data(endpoint, parameters_1925, Frost_Met_ID)

#Show the first 5 rows and last 5 rows of the average temperature DataFrame to v
df_1925.head(5)
df_1925.tail(5)
```

```
Data retrieved from frost.met.no!
```

Out[57]:

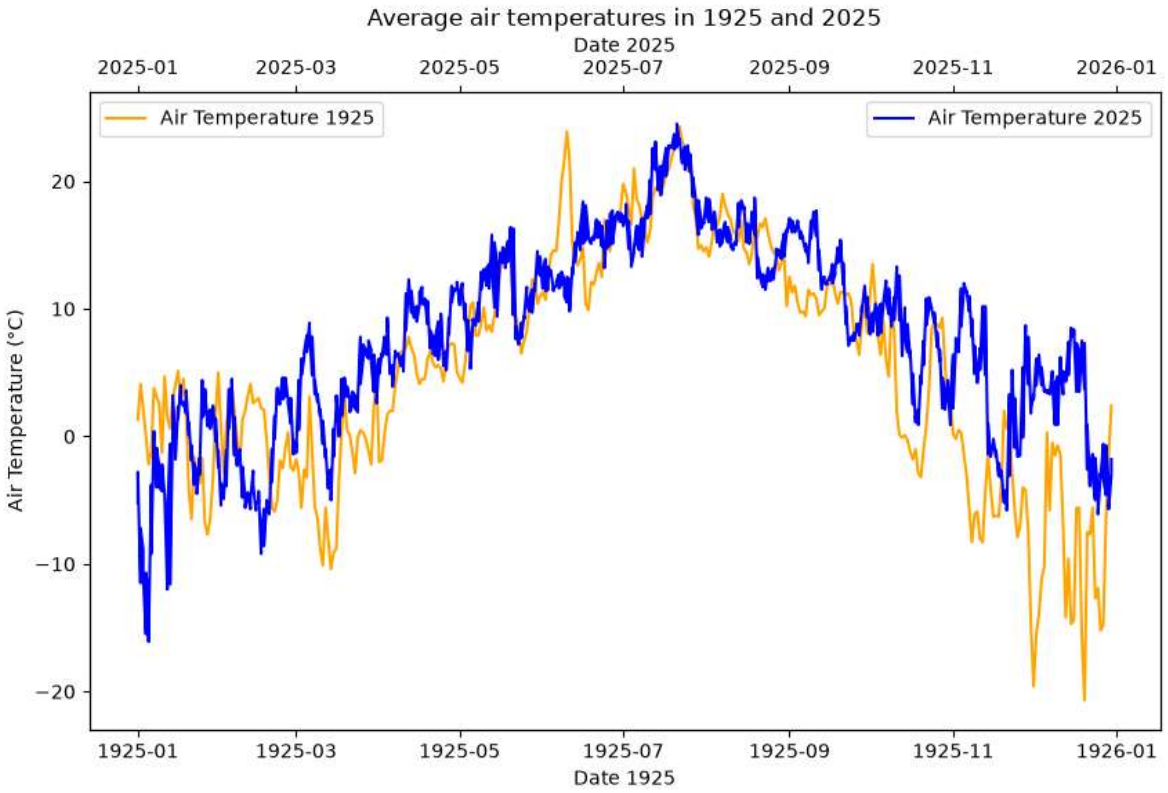
	index	elementId	value	unit	level	timeOffset	timeR
359	0	mean(air_temperature P1D)	-15.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	
360	0	mean(air_temperature P1D)	-14.8	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	
361	0	mean(air_temperature P1D)	-6.6	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	
362	0	mean(air_temperature P1D)	-1.5	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	
363	0	mean(air_temperature P1D)	2.4	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	

In [58]:

```
#Plot the air temperature data for 1925 and 2025 using matplotlib library.
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(df_1925['referenceTime'], df_1925['value'], label='Air Temperature 1925'
ax.set_xlabel('Date 1925')
ax.set_ylabel('Air Temperature (°C)')
ax.legend(loc='upper left')
ax.set_ylim(-23, 27)

#twiny means that the graphs share a common y-axis.
ax=ax.twiny()
ax.plot(temp_df['referenceTime'], temp_df['value'], label='Air Temperature 2025'
ax.set_xlabel('Date 2025')
ax.legend(loc='upper right')

plt.title('Average air temperatures in 1925 and 2025')
plt.show()
```



The graphs are plotted nicely with titels and labels. This plot makes sense because the graphs have roughly the same shape and the graph for 1925 lies under the 2025 for most of the dates meaning that the average temperature for 1925 is lower than for 2025.

Summary table for 1925 and 2025

In [59]:
```
#Calculate the mean, median, min, and max of the average air temperature data fo
Mean_air_temperature_2025 = temp_df[temp_df['elementId'] == 'mean(air_temperatur
Median_air_temperature_2025 = temp_df[temp_df['elementId'] == 'mean(air_temperat
Min_air_temperature_2025 = temp_df[temp_df['elementId'] == 'mean(air_temperature
Max_air_temperature_2025 = temp_df[temp_df['elementId'] == 'mean(air_temperature
Mean_air_temperature_1925 = df_1925[df_1925['elementId'] == 'mean(air_temperatur
Median_air_temperature_1925 = df_1925[df_1925['elementId'] == 'mean(air_temperat
Min_air_temperature_1925 = df_1925[df_1925['elementId'] == 'mean(air_temperature
Max_air_temperature_1925 = df_1925[df_1925['elementId'] == 'mean(air_temperature

#Store the calculated values in a pandas DataFrame.
df_summary = pd.DataFrame({'Value': [Mean_air_temperature_2025, Median_air_tempe
                                     Mean_air_temperature_1925, Median_air_tempera
                           index=['Mean Air Temperature 2025', 'Median Air Temperatur
                                  'Mean Air Temperature 1925', 'Median Air Temperatur

#print the DataFrame containing the calculated values.
print(df_summary)
```

```
                            Value
Mean Air Temperature 2025    7.90
Median Air Temperature 2025  8.35
Min Air Temperature 2025   -16.10
Max Air Temperature 2025    24.50
Mean Air Temperature 1925    5.31
Median Air Temperature 1925  5.30
Min Air Temperature 1925   -20.70
Max Air Temperature 1925    24.30
```

We see that the data is written in a table and the values are reasonable compered to the plot for 1925 and 2025 temperatures.

Task 4

In [60]:
```python
def number_of_heatwaves(
    endpoint: str,
    parameters: dict,
    ID: str,
    year: int,
    station_id: str,
    station_name: str = "Ås"
) -> dict:
    """This function retrives the maximum air temprature data
    from any station using the provided
    endpoint, parameters, and your ID for the station.
    The function writes a summary to a yml file containing the dates of the heat
    Returns the summary as a dictionary
    """
    #Update the parameters to get the correct station and timeperiod
    para = parameters.copy()
    para['sources'] = station_id
    para['referencetime'] = f"{year}-01-01/{year}-12-31"

    #Retrieve data by using an excisting function in this code
    df = get_met_data(endpoint, para, ID)

    if df.empty:
        print("No data to analyse")
        return{}

    #Filtering out only neccesary max-temperatures
    if 'elementId' in df.columns:
        df_temp = df[df['elementId'] == 'max(air_temperature P1D)'].copy()
    else:
        df_temp = df.copy()

    #Make sure the dates are sorted correctly
    df_temp['referenceTime'] = pd.to_datetime(df_temp['referenceTime'])
    df_temp = df_temp.sort_values('referenceTime').reset_index(drop = True)

    #Convert 'value' to a numerical value
    df_temp['value'] = pd.to_numeric(df_temp['value'], errors = 'coerce')

    #Mark days with temperature >= 27 degrees celsius
    df_temp['is_hot'] = df_temp['value'] >= 27

    #Usa a cumulative sum to group consecutive hot days
    df_temp['streak_group'] = (df_temp['is_hot'] != df_temp['is_hot'].shift()).c

    heatwaves = []

    #loop through the groupss tp find the groups that lasts for 5 days or more
    for _, group in df_temp.groupby('streak_group'):
        if group['is_hot'].iloc[0] == True and len(group) >= 5:
            start_date = group['referenceTime'].min().strftime('%Y-%m-%d')
            end_date = group['referenceTime'].max().strftime('%Y-%m-%d')
            heatwaves.append(f"{start_date} - {end_date}")
```

```
    #Build the structure for the yml-file
    yaml_data = {
        'dates': heatwaves,
        'station_id': station_id,
        'station_name': station_name,
        'year': year
    }

    #Write the summary to the yml-file with the correct format
    filename = f"heatwaves_summary_{station_id}_{year}.yml"
    with open(filename, 'w', encoding = 'utf-8') as f:
        yaml.dump(yaml_data, f, allow_unicode = True, sort_keys = False)

    print(f"Summary saved to {filename}")
    return yaml_data

endpoint = 'https://frost.met.no/observations/v0.jsonld'

parameters_max = {
    'sources': 'SN17850',
    'elements': 'max(air_temperature P1D)',
    'referencetime': '2025-01-01/2025-12-31',
}


#Call the function for Ås (SN17850) for 2025
result= number_of_heatwaves(
    endpoint = endpoint,
    parameters = parameters_max,
    ID  = Frost_Met_ID,
    year = 2025,
    station_id = "SN17850",
    station_name = "Ås"
)
```

```
Data retrieved from frost.met.no!
Summary saved to heatwaves_summary_SN17850_2025.yml
```

I called the function that counts the numbers of heatwaves per year for Ås (SN17850) for 2025 and 2018 to test if the function works. And the function works because the generated yml.file said it was one heatwave in 2025 in july and 4 heatwaves in 2018.

Task5 Function for moving avrage temp

In [61]:
```
#Defining a function that calculate a moving avrage fore the values of a pandas
def moving_average(df: pd.DataFrame, window_size: int) -> pd.DataFrame:
    """
    This function calculates the moving average of
    the data using a specified window size.
    It takes a pandas DataFrame containing the data
    and an integer representing the window size as input.
    It uses loops and boolean conditions to calculate the
    moving average for each data point in the DataFrame.
    It returns a pandas DataFrame containing
    the moving average of the data.
    """

    #Creat an empty list to contain the moving avrages
```

```
    moving_avg = []
    #start a loop to iterate through each index of the DataFrame
    for i in range(len(df)):

        #Check if the current index is less than or equal to the window/2
        if i <= window_size/2:
            #Create a window of values from index 0 to the current index
            window = df['value'][0:i+math.floor(window_size/2)]
            #Calculate the mean of the values in the window
            avrage = window.mean()
            #append the calculated moving average to the list
            moving_avg.append(avrage)

        #Take the current index since it is greater than the window/2
        else:
            #Create a window of values from the current index minus the window s
            window = df['value'][i - math.floor(window_size/2):i + math.floor(wi
            #Calculate the mean of the values in the window
            avrage = window.mean()
            #append the calculated moving average to the list
            moving_avg.append(avrage)
    #Create a new DataFrame to store the moving averages and the corresponding r
    moving_avg_df = pd.DataFrame(moving_avg, columns=['moving_average'])
    moving_avg_df['referenceTime'] = df['referenceTime']

    #Return the DataFrame for the moving avrages
    return moving_avg_df

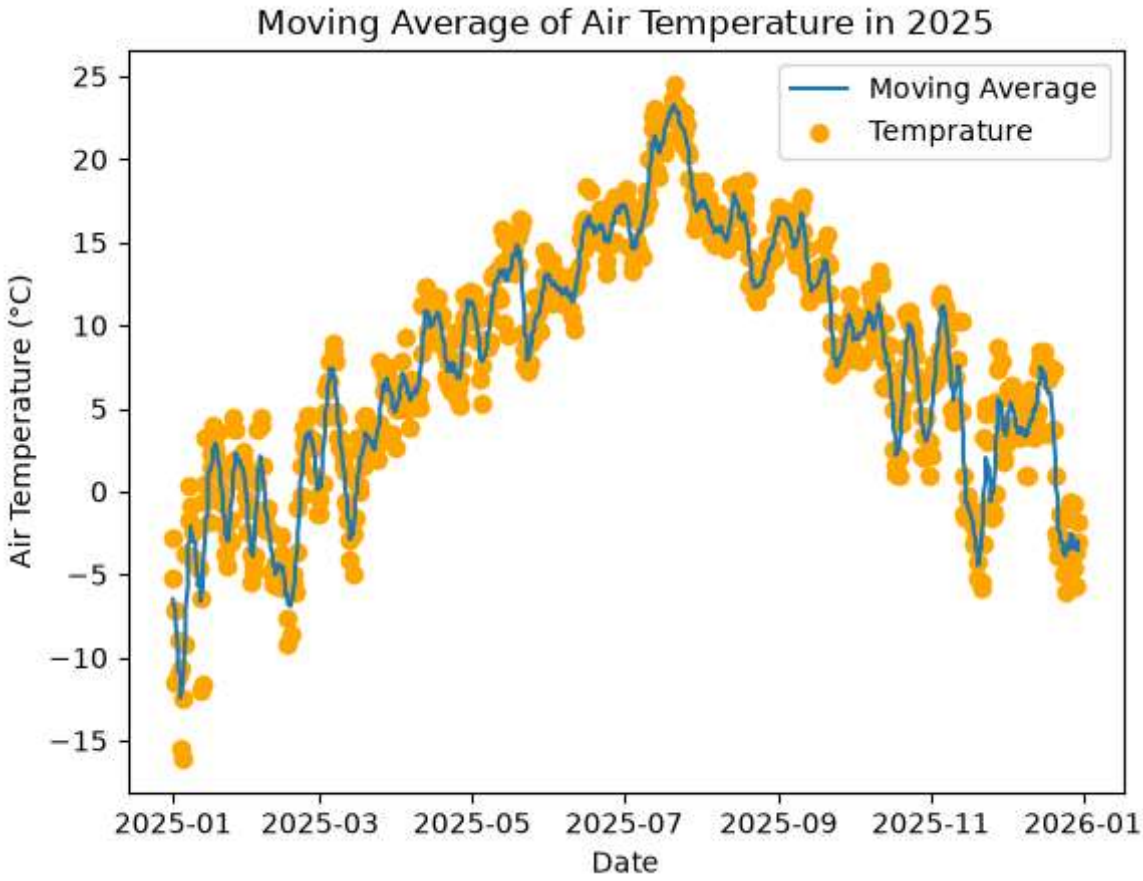
#Make a moving avrage dateframe using the moving_average function with a window
moving_avg_df = moving_average(temp_df, 7)

#print the moving average DataFrame to verify the calculated values.
print(moving_avg_df)

#plot the moving average of the air temperature data for 2025.
plt.plot(moving_avg_df['referenceTime'], moving_avg_df['moving_average'], label=
plt.scatter(temp_df['referenceTime'], temp_df['value'], label='Temprature', colo
plt.title('Moving Average of Air Temperature in 2025')
plt.xlabel('Date')
plt.ylabel('Air Temperature (°C)')
plt.legend()
plt.show()#shows the funktion
```

```
     moving_average             referenceTime
0         -6.500000 2025-01-01 00:00:00+00:00
1         -6.675000 2025-01-01 00:00:00+00:00
2         -7.120000 2025-01-02 00:00:00+00:00
3         -7.766667 2025-01-02 00:00:00+00:00
4         -9.483333 2025-01-03 00:00:00+00:00
..              ...                       ...
723       -3.083333 2025-12-28 00:00:00+00:00
724       -3.483333 2025-12-29 00:00:00+00:00
725       -3.233333 2025-12-29 00:00:00+00:00
726       -2.960000 2025-12-30 00:00:00+00:00
727       -3.525000 2025-12-30 00:00:00+00:00

[728 rows x 2 columns]
```



We se that the normal avrage is more scattered than the moving avrage. We than can say we calculate the moving avrage corektly because it is supose to make the graf more smooth.